Git Leak Hunter: A Defensive Full-History Secret-Exposure Scanner for Git Repositories and CI/CD Configuration

Pruthvi Gangapure
Roll No. 12415034

Prathamesh Yewale
Roll No. 12413969

Yash Waghmare
Roll No. 12410801

Chetan Thakur
Roll No. 12413081

Guide: Prof. Jyoti Yadav
Ethical Hacking Course Project, Group 34

***Abstract*—Credentials committed to a Git repository remain recoverable from its history even after a later commit deletes them, and CI/CD configuration files are a frequent, under-scanned place for such literals. This report presents Git Leak Hunter, a defensive command-line scanner that walks the *entire reachable history* of a repository, detects secrets with a hybrid of vendor-specific signatures and length-normalised Shannon entropy, parses GitHub Actions, GitLab CI and Dockerfile configuration structurally, classifies every finding with a table-driven severity function, and reports whether each secret is still present at `HEAD` or exists only in history. Detection results are memoised by blob hash, so the work done is proportional to the number of unique blobs rather than to commits × files. Optional live verification is opt-in, read-only, and limited to two allowlisted endpoints. No output ever contains a full secret. The implementation consists of 4,545 lines of TypeScript with a 306-test suite. On the project's eight synthetic fixture repositories (32 planted fabricated secrets) it reported all 32 with no false positives, against 28 for Gitleaks 8.30.1 and 14 for TruffleHog 3.99.3 in their default configurations; we discuss why this home-field result does not measure real-world accuracy. On a synthetic history with high blob reuse, memoisation reduced the median scan time of a 4,000-commit repository from 1084 ms to 164 ms.**

***Index Terms*—secret detection, credential leakage, Git history, software supply chain, CI/CD security, Shannon entropy, memoization, ethical hacking**

I. Introduction

Hard-coded credentials (CWE-798 [1]) are one of the most common ways that cloud accounts, package registries and CI/CD systems are compromised. Large-scale measurement shows that secret leakage in public repositories is widespread and that leaked keys are often not removed [2]. The usual remediation, deleting the secret in a new commit, does not help: Git is a content-addressed object store, and every earlier version of every file remains reachable from the commit graph until history is rewritten [3]. A scanner that inspects only the working tree therefore misses exactly the secrets a developer believes are gone.

CI/CD pipelines widen the problem. Workflow files, pipeline variables and container build arguments routinely carry tokens, and insufficient credential hygiene is listed among the top CI/CD security risks [4]. Compromise of a single CI credential can cascade through the software supply chain, as the 2021 Codecov Bash-uploader incident demonstrated [5].

Git Leak Hunter is a defensive tool built for repository owners. Given a local path or a remote URL, it answers four questions for every committed secret: *what* it is, *where* and in *which commit* it was introduced, *how severe* the exposure is, and *whether it is still present* at `HEAD` or recoverable only from history.

A. Contributions

- A four-stage pipeline (Detect → Analyze → Verify → Report) that walks every commit on every ref, including each parent side of merge commits, and reads content by blob hash without checking anything out.
- Blob-hash memoisation that reduces detection work from $O(\text{commits} \times \text{files})$ to $O(\text{unique blobs})$, verified by a reachable-blob invariant test.
- A hybrid detector combining nine signature rules with a length-normalised entropy threshold, thirteen counted suppression reasons, and structural parsers for GitHub Actions, GitLab CI, Dockerfiles and eight kinds of credential file.
- A safety design in which no data structure that reaches an output carries a raw secret, enforced by a JSON schema with `additionalProperties: false` and leak tests over every output channel.
- Hardening for the fact that the scanned repository is untrusted input: no hooks, no submodule recursion, NUL-delimited parsing, ReDoS-bounded detectors, and an XSS-safe report.
- A reproducible evaluation against Gitleaks [6] and TruffleHog [7] with an explicit list of the comparison's limitations.

The rest of this report is organised as follows. Section II reviews related work. Section III states the threat model and design rules. Sections IV and V describe the architecture and its hardening. Section VI covers the

implementation, Section VII the evaluation, Section VIII limitations, and Section IX responsible use.

II. Background and Related Work

A. Secret leakage in source repositories

Meli *et al.* [2] scanned public GitHub at scale and found that leaked API keys and private keys are common, that many remain in place, and that leaks appear in history as well as in the current tree. Sinha *et al.* [8] studied the detection and mitigation of API-key leaks in source repositories. Basak *et al.* released SecretBench [9], a labelled dataset of software secrets for evaluating detectors, and surveyed practitioner practices for secret management [10]. These studies motivate two design choices of Git Leak Hunter: scanning full history and treating detector precision as a first-class concern.

B. Existing scanners

Gitleaks [6] is a regex-based scanner for Git repositories, files and standard input. TruffleHog [7] combines a large detector library with optional live verification. GitHub secret scanning [11] runs provider-defined patterns on hosted repositories. Git Leak Hunter differs in emphasis rather than in kind: it (i) parses CI/CD files structurally instead of as plain text, (ii) reports *presence* (in-`HEAD` versus history-only) for every finding, (iii) diffs merge commits against every parent so that secrets introduced only during conflict resolution are found, and (iv) is built so that a full secret cannot reach any output.

C. Shannon entropy

For a string X over an alphabet with empirical symbol frequencies $p(x_i)$, Shannon entropy [12] in bits per character is

$$H(X) = -\sum_i p(x_i)\,\log_2 p(x_i). \tag{1}$$

Randomly generated tokens have higher H than identifiers and prose, which makes H a common second-stage filter for candidates that no vendor signature recognises.

III. Threat Model and Design Rules

A. Assets and adversaries

The asset is any credential committed to a repository the user owns or is authorised to assess. The tool must withstand two adversaries. The first is an outside attacker who mines public history for credentials; the tool's purpose is to find those credentials first. The second is the *scanned repository itself*: file names, paths, commit messages, author names, Git configuration and file contents are attacker-controlled input that may try to execute code, inject Git options, forge parser boundaries, exhaust resources, or inject script into the HTML report.

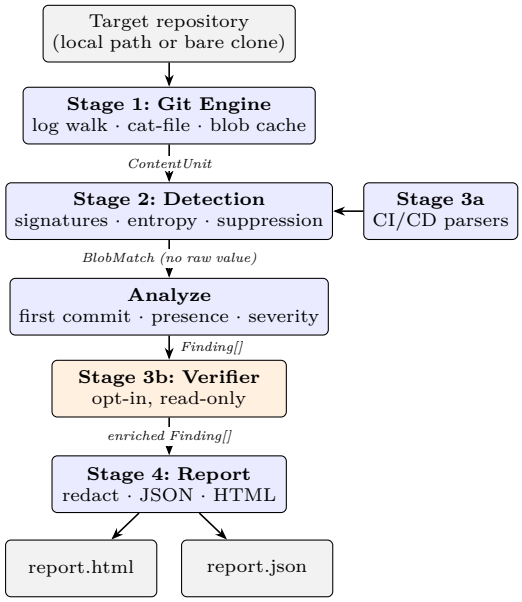


Fig. 1. The Detect→Analyze→Verify→Report pipeline. Raw secret values never cross the Detection→Analyze boundary.

B. Design rules

The project's working rules (`CLAUDE.md`) translate into six binding requirements:

- R1 **No full secret in any output**: HTML, JSON, console, logs, errors or test output.
- R2 **No fabricated numbers**: every reported figure comes from a script that ran, with tool versions, commands, machine and date recorded.
- R3 **Verification is opt-in, read-only and allowlisted**: it needs both `--verify` and `--confirm-authorized`, and only `sts.amazonaws.com` and `api.github.com` may be contacted.
- R4 **Fabricated fixtures only**: test secrets are generated at test time from fragments, and no token-shaped literal is ever committed.
- R5 **Scanned repositories are untrusted**: Git is spawned with an argument vector, never a shell string; hooks are disabled and nothing from the target is executed.
- R6 **No feature creep**: features beyond the specification require approval.

C. Non-goals

The tool does not exploit, enumerate or store discovered credentials, does not rewrite history, and does not scan repositories the user is not authorised to scan.

IV. System Architecture

Fig. 1 shows the pipeline, and Table I the data model `Repository` → `ContentUnit` → `Finding` → `Report`.

TABLE I
DATA MODEL (SRC/TYPES.TS)

Type	Meaning
`ContentUnit`	One file version as seen in one commit: repository, commit hash, author, date, path, blob hash, content, origin (`history`/`head`), line offsets.
`Finding`	One secret at one location: location-based fingerprint, rule, detector, severity, path, commit, line, column, *redacted* preview and context, entropy, first-seen commit, other-commit count, presence, verification status. **No raw-secret field.**
`Report`	Repository, scan options, measured statistics, severity summary and sorted findings; serialised as `report.json` and rendered as `report.html`.

TABLE II
SIGNATURE RULES (SRC/DETECT/SIGNATURES.TS)

Rule	Pattern (summary)
AWS access key ID	`AKIA`/`ASIA`/`AIDA` + 16 `[A-Z0-9]`, not inside a longer token
AWS secret key	40 chars `[A-Za-z0-9/+]` within 5 lines of a key ID, not all-hex, must pass entropy
GitHub token	`ghp_` `gho_` `ghu_` `ghs_` `ghr_` + 36 alphanumerics
GitHub fine-grained	`github_pat_` + 22 + `_` + 59 alphanumerics
Stripe live / test	`sk_live_`/`rk_live_`, `sk_test_`/`rk_test_` + ≥16 alphanumerics
Slack token	`xox[abprs]-` + dash-separated segments
Slack webhook	`https://hooks.slack.com/services/...`
Private key	PEM `BEGIN ... PRIVATE KEY` header through the matching footer (or 200 lines)

A. Stage 1: Git engine

The engine opens a local repository, or bare-clones a remote URL into a temporary directory with an empty template directory (so no hooks are installed) and no checkout. It then streams

```
git log --topo-order --reverse -m --root --raw
-z --no-abbrev --no-renames --no-ext-diff
--no-textconv --all
```

which yields, for every commit, its metadata and the list of changed paths with their new blob hashes [13]. The `-m` flag diffs each merge commit against *each* parent, so a secret introduced only while resolving a conflict is still seen. Content is read by blob hash through a single long-lived `git cat-file --batch-command` process [14], which is first asked for object `info` so that oversize blobs are never transferred. No commit is ever checked out.

Streaming parser. Output is split into NUL-terminated tokens as it arrives. Each commit header begins with a marker that carries a random per-run nonce, so an attacker-chosen commit subject or file name cannot forge a commit boundary. Memory therefore grows with the number of file versions, not with blob content.

Memoisation. A naive scanner reads every file of every commit, i.e. $O(\text{commits} \times \text{files})$ work. Git Leak Hunter reads only the files each commit changed and caches detection results under the key

$$k = \mathit{configHash} \,\|\, \mathit{blobHash} \,\|\, \mathit{fileKind}, \tag{2}$$

so identical content reached through other commits, paths, branches or reverts is scanned once, giving $O(\text{unique blobs})$ detection work [15]. The file kind is part of the key because the same bytes are parsed differently as a workflow file and as documentation. Severity depends on path class and presence, so it is applied after the cache. Tests assert that the scanner function runs exactly once per unique (blob, kind) and that

$$\mathit{uniqueBlobsScanned} + \mathit{cacheHits} = \mathit{fileVersionsSeen}. \tag{3}$$

Reachable-blob invariant. For every fixture repository, with and without ignore rules, a test checks that

$$S_{\text{scanned}} \cup S_{\text{skipped}} \cup S_{\text{ignored}} = S_{\text{rev-list}}, \tag{4}$$

where $S_{\text{rev-list}}$ is the set of blobs listed by `git rev-list --objects --all`. The test fails if any commit-reachable blob, including one that exists only in a merge resolution, was never considered.

Presence. After the walk, the `HEAD` tree (or the `--branch` tip) is listed. A finding is `in-HEAD` if the same secret is found at the same path there, and `history-only` otherwise.

B. Stage 2: Detection engine

1) Candidate generation: Candidates come from three sources: vendor signatures (Table II), structured parsing of CI/CD and credential files (Section IV-C), and generic `key = value` assignments whose key name looks secret-like (`api_key`, `secret`, `token`, `password`, ...). Overlapping spans are resolved by priority signature > credential file > CI > entropy.

Each AWS key ID is paired with the nearest unclaimed 40-character candidate within five lines; both findings record each other's fingerprint, which is what live verification needs. A private-key block is one finding at its header line, with a preview such as "`BEGIN RSA PRIVATE KEY [REDACTED, 14 lines]`"; the PEM dashes are dropped deliberately so that a report never looks like a key file to another scanner.

2) Length-normalised entropy: A string of length n cannot exceed $\log_2 n$ bits per character, and random strings measure well below that bound. A fixed 4.5-bit threshold for base64-like strings is therefore unreachable for 20–22-character strings and rejects most random 24–32-character secrets (Table XI). Git Leak Hunter classifies each candidate's charset as hex, base64-like or other, with alphabet size $A \in \{16, 64, 94\}$, and uses the effective threshold

$$T_{\text{eff}} = \min\Big(T_{\text{charset}},\ \lambda \log_2 \min\big(\max(n, n_{\min}),\ A\big)\Big) \tag{5}$$

with $\lambda = 0.85$, defaults $T_{\text{hex}} = 3.0$, $T_{\text{base64}} = 4.5$ and $n_{\min} = 20$. Long strings are held to the configured threshold, and shorter strings must reach 85% of the maximum entropy a string of that length could have. Normalising

at $\max(n, n_{\min})$ rather than n prevents short words from passing: `postgres1` (9 characters) has $H = 2.948$, above $0.85 \log_2 9 = 2.69$, but below the threshold for a 20-character string.

3) False-positive suppression: Thirteen suppression reasons are implemented and each dropped candidate increments a counter in the report: inline `leakhunter:ignore`, documentation example keys, repeated characters, placeholders (`changeme`, `your_api_key_here`, ...), CI expressions (`${{ ... }}`), environment references (`$VAR`, `${VAR}`, `%VAR%`, `process.env.X`, ...), RFC 4122 UUIDs [16], 40-hex Git SHAs, lockfile integrity hashes, data URIs, vendored paths, low entropy, and fingerprints listed in an ignore file. Signature matches are exempt from the UUID, SHA and lockfile rules, so a real token inside a lockfile is still reported.

C. Stage 3a: CI/CD and credential files

The file kind is derived from the path and is part of the cache key, so all of the following are scanned across history, not only at `HEAD`:

- **GitHub Actions**: literal scalars under `env:` (workflow, job, step, container, services), `with:` inputs and container `credentials:`.
- **GitLab CI**: global and job `variables:`, including `{ value: ... }` and `include: local` files.
- **Dockerfile**: `ENV K=V`, legacy `ENV K V` and `ARG K=V`, with backslash continuations joined through an offset map.
- **Credential files**: `.env`, `.npmrc`, `credentials*.json`, `.docker/config.json`, SSH private-key files, `.pgpass`, `.netrc` and `.git-credentials`.

YAML is parsed into an AST so that source offsets give exact line and column. Malformed YAML raises a parse error and the file is re-scanned as text, so a broken file never aborts a scan. A literal under a secret-like key that fails the entropy check but is at least eight characters long is reported as a *weak* literal (Low).

D. Analyze: attribution and severity

`analyze.ts` groups matches into findings (one secret, one path, one rule), using a keyed HMAC of the value with a per-run random key that is never written anywhere. It records the commit that first introduced the secret at that path, how many other commits carried it, and presence. The fingerprint is the first 32 hex characters of SHA-256(*ruleId*, *blobHash*, *path*, *line*, *column*): it is location-based, stable across runs, and never derived from the secret.

Severity is a pure, table-driven function in which the first matching row wins (Table III). Per-rule overrides may be set in a configuration file, but a credential verified as active is always Critical.

E. Stage 3b: Optional live verification

Verification runs only when *both* `--verify` and `--confirm-authorized` are given, and an authorised-use banner is printed before any request. Exactly one read-only call is made per unique credential:

TABLE III
SEVERITY TABLE (`SRC/DETECT/SEVERITY.TS`); FIRST MATCH WINS

#	Severity	Condition
1	Critical	Live verification returned `ACTIVE`
2	Critical	Private key still present at `HEAD`
3	Critical	Stripe live key still present at `HEAD`
4	Low	Secret-like key with a low-entropy literal
5	High	Signature match for AWS, GitHub, Slack, live Stripe or a private key (incl. history-only)
6	High	Literal secret in GitHub Actions, GitLab CI or a Dockerfile
7	High	Committed credential file (`.env`, `.npmrc`, `id_rsa`, ...)
8	Medium	Stripe test key
9	Low	Entropy-only candidate in tests, docs, examples or fixtures
10	Medium	Entropy-only candidate in application code
11	Low	Anything else

- **AWS**: `POST https://sts.amazonaws.com/` with `Action=GetCallerIdentity` [17], signed with AWS Signature Version 4 [18] using the paired key ID and secret. HTTP 200 means `ACTIVE`; `InvalidClientTokenId`, `SignatureDoesNotMatch`, `ExpiredToken` or `AccessDenied` mean `INACTIVE`; anything else (including an unpaired key or temporary `ASIA` credentials) is `UNVERIFIABLE`.
- **GitHub**: `GET https://api.github.com/user` [19]; 200 is `ACTIVE`, 401 is `INACTIVE`, rate limiting and other outcomes are `UNVERIFIABLE`.

Guardrails: an exact-host, HTTPS-only allowlist checked before any request; no redirects; a 10 s timeout; no retries; sequential calls 250 ms apart, at most 25 per provider; and a stop at the first rate-limit response. The GitHub response body is never read, and from STS only the error code is read, so no account identifiers are retained. SigV4 is implemented in about 60 lines rather than by adding the AWS SDK and its dependency tree, and is tested against AWS's published signing vectors.

When verification is on, raw values of verifiable rules are placed in an in-memory *vault*; each match carries only an opaque random reference. The vault is cleared in a `finally` block as soon as verification ends, and its `toJSON` prints only an entry count.

F. Stage 4: Reporting and redaction

Redaction. Values of at least 12 characters are shown as the first 4 characters, 16 stars and the last 4 (e.g. `AKIA****************WXYZ`); the star count is fixed so the preview does not reveal the length. Shorter values show at most 2 leading characters and 8 stars. Context lines are masked twice: first every detected span, then any residual token-like run of at least 16 characters that mixes letters

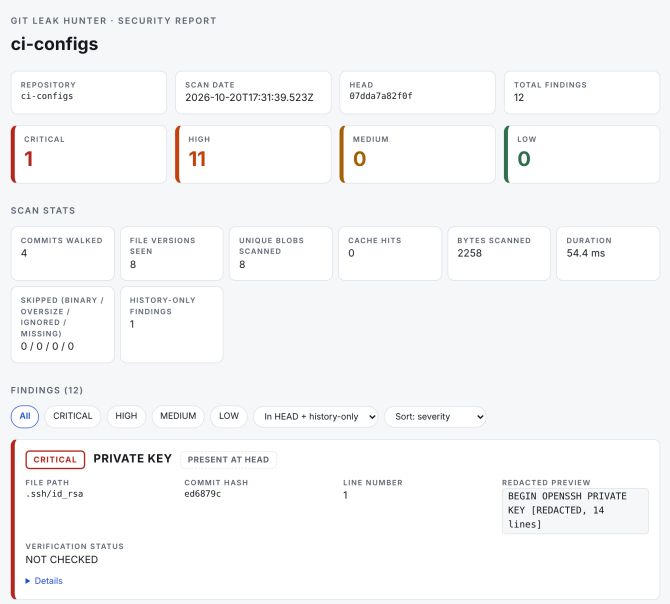


Fig. 2. Top of `examples/report.html`, generated from the `ci-configs` fixture (fabricated secrets only). Severity counts, measured scan statistics and the first finding card with its redacted preview and presence badge are visible.

and digits or is mixed-case with entropy ≥ 3.5, so that an undetected secret on the same line cannot leak. Commit subjects and author fields, which are attacker-controlled and may contain pasted secrets, pass through the same masking.

JSON. `report.json` is validated in the tests against a JSON Schema (draft 2020-12 [20]) that uses `additionalProperties: false` everywhere, so a stray field, such as an accidentally added raw value, fails validation.

HTML. `report.html` is a single self-contained file with inline CSS and one inline script for filtering by severity and presence and for sorting. It loads no external resources and sets the Content Security Policy [21] `default-src 'none'; style-src 'unsafe-inline'; script-src 'unsafe-inline'`. Every dynamic value is HTML-escaped and control characters in paths are shown as `\xNN`. Fig. 2 shows a report generated from the `ci-configs` fixture.

G. Optional: owner-scoped GitHub dorking

The `dork` sub-command runs built-in code-search terms (`AKIA`, `ghp_`, `filename:.env`, ...) through the GitHub code-search API [22] and feeds the matching files into the same detection and report pipeline. It is restricted to owners the user controls: before any search the token's account must be the owner itself, or an active admin of the organisation. Every query carries exactly one `user:`/`org:` qualifier added by the tool, queries containing any other scope qualifier or `OR` are refused before the first request, out-of-owner results are dropped, and requests are paced to the 10-per-minute search budget, stopping at the first rate limit.

V. Hardening Against Untrusted Input

Table IV summarises how the threats of Section III are mitigated and which tests cover them. Every Git invocation goes through one module (`src/git/exec.ts`), always with an argument vector, and is prefixed with

```
-c core.hooksPath=/dev/null  -c core.fsmonitor=false
-c core.pager=cat            -c protocol.allow=never
-c protocol.https.allow=always
-c protocol.ssh.allow=always -c submodule.recurse=false
```

Variables that would redirect Git (`GIT_DIR`, `GIT_WORK_TREE`, ...) are removed from the environment and terminal prompts are disabled.

VI. Implementation

Git Leak Hunter is written in TypeScript as an ES-module Node.js command-line tool (runtime Node ≥ 20, Git ≥ 2.36). It has three runtime dependencies: `commander` (argument parsing), `picocolors` (terminal colour) and `yaml` (YAML AST). Table V gives the size of each module.

A. Command-line interface

```
git-leak-hunter scan <path-or-url>
  [--out dir] [--format html,json]
  [--max-commits n] [--since date] [--branch name]
  [--no-ci] [--no-cache] [--min-length n]
  [--entropy-hex n] [--entropy-base64 n]
  [--ignore file] [--config file]
  [--fail-on critical|high|medium|low]
  [--verify --confirm-authorized]
  [--quiet | --verbose]
git-leak-hunter dork --owner <login> [--org]
  --confirm-authorized
```

Exit code 0 means the scan completed with no finding at or above `--fail-on`, 1 means such a finding exists, and 2 signals a usage or runtime error, which makes the tool usable as a CI gate. Ignore rules can come from `.leakhunterignore` at the scanned `HEAD`, `--ignore`, the configuration file, or an inline `leakhunter:ignore` comment; the configuration file itself is never read from the scanned repository.

B. The project's own supply-chain hygiene

Because the tool targets supply-chain risk, the repository applies the same discipline to itself: dependencies are pinned exactly (`save-exact`), install scripts are disabled (`ignore-scripts`), new versions are adopted only after 14 days of publication, the CI workflow runs with `contents: read` permissions and pins every action to a full commit SHA, and `npm audit` runs in CI. CI runs typecheck, lint, format check, tests, build and audit on Ubuntu and macOS with Node 22 and 24.

TABLE IV
THREATS FROM THE SCANNED REPOSITORY AND THEIR MITIGATIONS (`docs/ARCHITECTURE.md`)

Threat	Mitigation	Test
Hooks, fsmonitor, pager, `ext::`/`file://` transports, submodules	`-c` overrides on every Git call; remote URL allowlist (`https`, `ssh`)	`repo`, `cli`, `remote`
Environment variables redirecting Git; SSH prompts hanging a scan	Stripped from the environment; `GIT_TERMINAL_PROMPT=0`; `ssh -o BatchMode=yes`	`repo`
Option injection via refs or paths	`--end-of-options` for refs; only commit IDs passed to `log`/`ls-tree`; `--` before paths	`log`, `scan`
Forged commit boundaries in subjects or paths	NUL-separated tokens plus a random per-run marker nonce	`log`
Hostile file names (newline, tab, leading `-`, HTML)	`-z` everywhere; paths never passed as arguments; control chars shown as `\xNN`	`hostile` fixture
XSS in paths, author names, commit messages	Escape every dynamic value; strict CSP; no external resources	`scan`
Regular-expression DoS [23]	Identifier-boundary lookbehind, bounded key/value lengths, character-bounded AWS pairing window, bounded context, binary-search overlap resolution	`redos`
Huge files, binaries, missing objects	2 MiB cap checked with `cat-file info` before reading; NUL-byte sniff; missing objects counted	`hostile`, `invariant`
Malformed YAML	Falls back to text scanning	`ci`
Temporary clone left behind	`finally` cleanup plus synchronous exit hook; SIGINT/SIGTERM handled	`remote`
Credentials in clone URLs	Stripped from notices, errors and the report	`remote`
Large histories	Streaming log parser, one `cat-file` process, per-blob memoisation	`scale` (2,000 commits)

TABLE V
SOURCE SIZE BY MODULE (PHYSICAL LINES, `wc -l`, COMMIT `b6f983a`)

Directory	Responsibility	Lines
`src/*.ts`	CLI, pipeline, analyze, config, types	1,086
`src/git/`	Stage 1: exec, repo, log, cat-file	773
`src/detect/`	Stage 2: signatures, entropy, suppression	925
`src/ci/`	Stage 3a: CI/CD and credential-file parsers	518
`src/verify/`	Stage 3b: vault, HTTP allowlist, SigV4, providers	398
`src/report/`	Stage 4: redact, sanitize, JSON, HTML	366
`src/dork/`	Optional owner-scoped code search	479
Total, `src/`		**4,545**
`test/`	Unit, integration, fixtures, helpers	4,587
`bench/`	Benchmark and calibration scripts	810

TABLE VI
TEST COVERAGE (E2, `npx vitest run --coverage`, V8 PROVIDER)

Metric	Covered / total	Percent
Statements	1,551 / 1,659	93.49%
Branches	991 / 1,186	83.55%
Functions	231 / 256	90.23%
Lines	1,386 / 1,432	96.78%

C. Test fixtures

Eight fixture repositories are built at test time with real `git` commands from a seeded PRNG (Table VII). Vendor prefixes are assembled from fragments (e.g. `['gh', 'p_'].join("")`), so no token-shaped literal exists in the repository; a hygiene test scans the tree for seven token shapes and fails on any hit. The builder also writes `ground-truth.json` with each planted secret's file, commit, line and expected presence.

VII. EVALUATION

Following rule R2, each subsection names where its numbers come from. Two environments were used:

- **E1**, the committed benchmark results in `docs/BENCHMARK.md`: Apple M1, 8 cores, 8 GiB RAM, Darwin 27.0.0 arm64, Node v24.21.0, Git 2.54.0; runs on 2026-10-20 and 2026-10-21.
- **E2**, runs made for this report: Intel Xeon @ 2.10 GHz, 4 vCPUs, 15 GiB RAM, Linux 6.18 x86_64, Node v22.22.0, Git 2.43.0; runs on 2026-10-26 at commit `b6f983a`.

A. Test suite and coverage (E2)

`npm run typecheck` and `npm run lint` completed without errors. `npx vitest run` executed 24 test files with 306 tests, all passing, in 10.87 s of wall time. `npx vitest run --coverage` reported the coverage in Table VI. The suite includes positive and negative cases for every detector, the reachable-blob invariant, leak checks over HTML, JSON, stdout and stderr (using a helper that never prints the secret it searches for), XSS and schema checks, ReDoS time budgets on inputs that previously took up to about 65 s, a 2,000-commit synthetic repository, the remote-clone path, and mocked verification in which the global `fetch` fails the test if any non-allowlisted host is contacted.

B. Scenario results per fixture (E2)

Table VII lists each fixture and the console summary of `npx tsx src/cli.ts scan <fixture>` on E2, with fixtures built by `npm run fixtures` (seed 1337). Every repository produced exactly its planted count; the `false-positives` repository, which contains 23 baits (UUIDs, lockfile hashes, placeholders, `${{ secrets.X }}`, `.env.example`, repeated characters, ...), produced no findings with 10 candidates suppressed. On `blob-reuse`, one secret-bearing blob is copied to four paths, deleted, restored and branched; 15 file versions were served from 3 unique blobs and 12 cache hits.

Dogfooding. Scanning the project's own repository on E2 walked 8 commits and 163 file versions, suppressed 35 candidates, and reported 0 findings. A test asserts this on every run.

C. Accuracy comparison (E1)

Command. `npx tsx bench/accuracy.ts`, run on 2026-10-21 at commit `70f78a1`. Each tool ran once per fixture with its default rules, in a non-network mode, without live verification (full command lines are in `docs/BENCHMARK.md`):

- Git Leak Hunter: `scan <repo> --format json --quiet`;
- Gitleaks 8.30.1: `gitleaks git <repo> --redact`;
- TruffleHog 3.99.3: `trufflehog git file://<repo> --json --no-verification --no-update`.

Matching. A tool finding is a true positive (TP) when its (repository, path, line) equals a planted secret; repeated reports of one planted secret count once. Each distinct unmatched location is a false positive (FP); planted secrets never matched are false negatives (FN). Table VIII gives the results, and Table IX the per-repository breakdown. The F_1 column is computed from the measured TP, FP and FN.

Where the tools differ. Gitleaks missed both secrets in `merge-conflict` (one present only in a conflict resolution, one in a file added only in the merge commit) and two Dockerfile literals (`ARG` and legacy space-separated `ENV`). TruffleHog's 18 misses are the two merge-only secrets; 11 of the 12 secrets in `ci-configs` (CI and Dockerfile literals, credential files and the `id_rsa` key); the Stripe test key, the deploy key and two generic high-entropy values in `head-secrets`; and the AWS secret key in `history-only`. The private-key misses are an artefact of the fixtures: planted keys are random bytes in PEM armour, not parseable keys, and a manual check recorded in `docs/BENCHMARK.md` showed that TruffleHog 3.99.3 does report a freshly generated, never-used RSA key while ignoring a fabricated one.

Threats to validity. This comparison has a strong home-field advantage. The fixtures were written by the same team, alongside the tool, and its thresholds were tuned against them (Section VIII). A perfect score shows that the rules do what they were designed to do; it is *not* an estimate of real-world precision or recall. The corpus is small and synthetic, competitors ran with default configurations only, matching is by location so a match anchored on a different line counts as a miss plus a false positive, and some planted categories (Dockerfile and CI literals) are ones Git Leak Hunter parses structurally while other tools may not target them by default. An evaluation on an independent labelled corpus such as SecretBench [9] would be needed for a general accuracy claim.

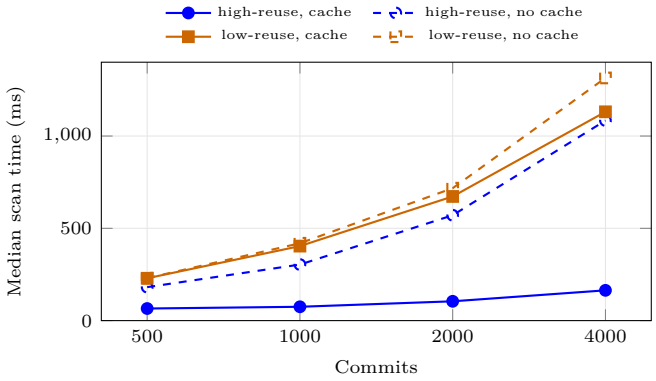


Fig. 3. Median scan time against history length (data of Table X, E1). In the high-reuse profile the cached scan grows slowly because the number of unique blobs stays at 150.

D. Memoisation benchmark (E1)

Command. `npx tsx bench/memoization.ts 5`, run on 2026-10-20 at commit `e4f3de8`. Synthetic repositories were generated with `git fast-import`: 300 paths, five files rewritten per commit, every tenth file content carrying a fabricated token (seed 2026). The *high-reuse* profile draws from a pool of 150 distinct contents; the *low-reuse* profile from up to 50,000, so almost every write is new. Each configuration had one unrecorded warm-up and five measured runs alternating cache on and `--no-cache`. Time is the engine's own `stats.durationMs` (Git walk, blob reads and detection; report writing excluded). Table X and Fig. 3 give the results.

In the high-reuse profile, the number of blobs on which detection runs stays at 150 while the file versions seen grow linearly with history, and the ratio of medians rises from 2.77× at 500 commits to 6.61× at 4,000 commits. In the low-reuse profile there is little to reuse and the two modes take similar time (1.00× to 1.16×): memoisation pays off in proportion to blob reuse (reverts, copies, branches, generated files) and does nothing for histories in which every change is unique. Even without the cache, the changed-files walk reads far fewer file versions than a snapshot-per-commit scanner would ("Snapshot file versions", a count obtained by summing `git ls-tree -r` entries over all commits; it is not a timing). These are single-machine timings of synthetic repositories and are not a prediction for any particular real repository.

TABLE VII
FIXTURE SCENARIOS AND SCAN RESULTS (E2, 2026-10-26). PLANTED = SECRETS IN `ground-truth.json`.

Fixture	Scenario	Planted	Commits	File versions	Unique blobs	Cache hits	Findings	C/H/M/L; history-only
`history-only`	AWS pair + GitHub token added, then removed	3	4	6	6	0	3	0/3/0/0; 3
`head-secrets`	Stripe, Slack, private key, generic high-entropy at `HEAD`	7	5	7	7	0	7	2/2/2/1; 0
`blob-reuse`	One secret blob at 4 paths, deleted/re-added, side branch	4	14	15	3	12	4	0/4/0/0; 2
`merge-conflict`	Secret only in a conflict resolution; file added only in the merge	2	4	7	5	2	2	0/2/0/0; 0
`ci-configs`	GitHub Actions, GitLab CI, Dockerfile, `.env`, `id_rsa`, `.npmrc`, `credentials.json`	12	4	8	8	0	12	1/11/0/0; 1
`false-positives`	23 baits: UUIDs, lockfile hashes, placeholders, CI expressions, …	0	4	13	13	0	0	–
`hostile`	XSS in names/author/message; newline, tab, leading-dash paths; huge file; binary	4	2	6	4	0	4	1/3/0/0; 0
`clean`	No secrets	0	2	5	5	0	0	–

TABLE VIII
ACCURACY ON 32 PLANTED SECRETS (E1, `bench/results/accuracy.json`). GLH = GIT LEAK HUNTER

Tool	TP	FP	FN	Prec.	Recall	F_1	Raw
GLH 0.1.0	32	0	0	100.0%	100.0%	1.000	32
Gitleaks 8.30.1	28	0	4	100.0%	87.5%	0.933	29
TruffleHog 3.99.3	14	0	18	100.0%	43.8%	0.609	15

TABLE IX
PER-REPOSITORY TP / FP / FN (E1)

Repository	Git Leak Hunter	Gitleaks	TruffleHog
`history-only`	3/0/0	3/0/0	2/0/1
`head-secrets`	7/0/0	7/0/0	3/0/4
`blob-reuse`	4/0/0	4/0/0	4/0/0
`merge-conflict`	2/0/0	0/0/2	0/0/2
`ci-configs`	12/0/0	10/0/2	1/0/11
`false-positives`	0/0/0	0/0/0	0/0/0
`hostile`	4/0/0	4/0/0	4/0/0
`clean`	0/0/0	0/0/0	0/0/0

E. Entropy calibration (E1)

Command. `npx tsx bench/entropy-calibration.ts 10000`, run on 2026-10-20 on E1 with thresholds hex 3.0, base64 4.5, $n_{\min} = 20$, $\lambda = 0.85$. Each cell of Table XI is the share of 10,000 seeded random strings (seed 2026) that *fail* the entropy check and would be missed. Length normalisation reduces the miss rate for 24-character alphanumeric strings from 95.21% to 1.67%, while the ordinary identifiers tested (e.g. `getUserAccountSettingsForTenant`, $H = 3.741$ against $T_{\text{eff}} = 4.211$) are still rejected. One tested string, the alphabet `abcdefghijklmnopqrstuvwxyz` ($H = 4.700$), passes the check, which illustrates why entropy is used only after a secret-like key name has been matched.

VIII. DISCUSSION AND LIMITATIONS

Tuning against the fixtures. Five rule changes were made after scanning the fixtures or the project's own repository, and are documented in `docs/RULES.md`: the length-normalised threshold; normalising at $\max(n, n_{\min})$; requiring values under non-secret-like keys to be at least $n_{\min}$ characters (which removed five false positives such as `DB_HOST=localhost` on `ci-configs`); excluding brackets from unquoted values; and suppressing values containing a `${...}` interpolation. These changes are the reason the fixture results in Section VII cannot be read as generalisation.

Known limitations (`docs/ARCHITECTURE.md`, `docs/RULES.md`):

- Blobs or trees reachable only through a ref or tag that points directly at a blob or tree, rather than at a commit, are not scanned, since no commit exists to attribute them to.
- At the default thresholds, roughly 1–3% of random 20–40-character secrets with no recognisable format fail the entropy check (Table XI). This affects only entropy-layer candidates, never signature matches.
- Presence is relative to `HEAD` (or the `--branch` tip); a secret that lives only on another branch is reported as `history-only`.
- `dork` sees only what GitHub code search indexes: default branches, files under its size limit, non-fork repositories. Its findings are leads to confirm with `scan <repo-url>`.
- Live verification covers long-term AWS keys with a paired secret and GitHub tokens only.
- The signature set covers AWS, GitHub, Stripe, Slack and private keys; other providers are detected only through the generic key-name and entropy path.

IX. ETHICAL CONSIDERATIONS AND RESPONSIBLE USE

Git Leak Hunter is a defensive tool for repository owners, and several of its rules are enforced in code rather than left

TABLE X
BLOB MEMOISATION: CACHE ON VS. `--no-cache` (E1, `bench/results/memoization.json`). TIMES ARE MEDIAN / MINIMUM OF FIVE RUNS.

Profile	Commits	Snapshot file versions	File versions seen	Unique blobs scanned	Cache hits	Cache (ms)	No cache (ms)	Ratio of medians
high-reuse	500	131,998	2,482	150	2,332	65.5 / 63.0	181.3 / 177.5	2.77×
high-reuse	1,000	281,998	4,964	150	4,814	74.9 / 73.9	303.6 / 301.1	4.05×
high-reuse	2,000	581,998	9,939	150	9,789	104.6 / 101.6	570.8 / 567.1	5.46×
high-reuse	4,000	1,181,998	19,865	150	19,715	164.0 / 162.1	1084.0 / 1078.2	6.61×
low-reuse	500	131,954	2,500	2,451	49	229.4 / 226.7	230.0 / 227.6	1.00×
low-reuse	1,000	281,954	5,000	4,772	228	403.7 / 397.6	419.1 / 414.4	1.04×
low-reuse	2,000	581,954	10,000	9,079	921	672.6 / 665.6	717.4 / 712.2	1.07×
low-reuse	4,000	1,181,954	20,000	16,538	3,462	1131.3 / 1126.4	1314.8 / 1303.3	1.16×

TABLE XI
SHARE OF RANDOM STRINGS MISSED BY THE ENTROPY CHECK (E1, 10^4 STRINGS PER CELL). N = LENGTH-NORMALISED (SHIPPED), F = FIXED THRESHOLD.

Charset		16	20	24	28	32	40	48	64
alnum	N	25.41	1.15	1.67	1.71	2.02	2.56	0.04	0.00
	F	100.0	100.0	95.21	72.71	33.25	2.56	0.04	0.00
base64	N	23.80	1.11	1.43	1.48	1.58	1.83	0.04	0.00
	F	100.0	100.0	94.65	70.57	29.92	1.83	0.04	0.00
hex	N	14.77	2.94	0.55	0.11	0.00	0.00	0.00	0.00
	F	14.77	2.94	0.55	0.11	0.00	0.00	0.00	0.00

Column headings are string lengths n; cells are percentages.

to policy. Users must scan only repositories they own or are explicitly authorised to scan; an authorisation notice is printed before any remote clone. Test repositories use only fabricated, non-functional secrets, and a hygiene test fails if a token-shaped literal is committed. Live verification requires two explicit flags, makes one read-only call per credential to one of two allowlisted hosts, and discards account identifiers. The `dork` command refuses to search outside the token owner's own account or administered organisation. There is no telemetry: the only network access is a clone the user requests, opt-in verification, and `dork` when run.

When a live credential is found, the documented procedure is to stop, record only the redacted preview, path, commit and fingerprint, notify the owner privately, and let the owner rotate the credential. A secret found in history must be treated as compromised and rotated first; rewriting history with a tool such as `git filter-repo` [3] is secondary and does not undo the exposure.

X. CONCLUSION

Git Leak Hunter shows that a full-history, CI/CD-aware secret scanner can be built so that its safety properties are structural rather than procedural: findings have no raw-secret field, the report schema rejects unexpected fields, verification is opt-in and allowlisted, and the scanned repository is treated as hostile input throughout. Blob-hash memoisation makes the cost of a full-history walk depend on unique content rather than on commits × files, and the measured benchmark shows the effect growing with history length when content is reused. On the project's own fixtures the tool detected all planted secrets, including merge-only and CI/CD literals that the two compared tools missed in their default configurations; establishing real-world accuracy requires an independent labelled corpus, which is the most important next step for this work.

ACKNOWLEDGMENT

The authors thank Prof. Jyoti Yadav for guidance throughout this Ethical Hacking course project.

REFERENCES

[1] MITRE, "CWE-798: Use of hard-coded credentials," *Common Weakness Enumeration*. [Online]. Available: https://cwe.mitre.org/data/definitions/798.html

[2] M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? Characterizing secret leakage in public GitHub repositories," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2019.

[3] E. Newren, "git-filter-repo." [Online]. Available: https://github.com/newren/git-filter-repo

[4] OWASP Foundation, "OWASP top 10 CI/CD security risks." [Online]. Available: https://owasp.org/www-project-top-10-ci-cd-security-risks/

[5] Codecov, "Bash uploader security update," Apr. 2021. [Online]. Available: https://about.codecov.io/security-update/

[6] Gitleaks, "Gitleaks: Find secrets with Gitleaks." [Online]. Available: https://github.com/gitleaks/gitleaks

[7] Truffle Security, "TruffleHog." [Online]. Available: https://github.com/trufflesecurity/trufflehog

[8] V. S. Sinha, D. Saha, P. Dhoolia, R. Padhye, and S. Mani, "Detecting and mitigating secret-key leaks in source code repositories," in *Proc. 12th IEEE/ACM Working Conf. Mining Softw. Repositories (MSR)*, Florence, Italy, 2015, pp. 396–400.

[9] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "SecretBench: A dataset of software secrets," in *Proc. 20th IEEE/ACM Int. Conf. Mining Softw. Repositories (MSR)*, Melbourne, Australia, 2023, pp. 347–351.

[10] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "What are the practices for secret management in software artifacts?" in *Proc. IEEE Secure Develop. Conf. (SecDev)*, 2022.

[11] GitHub, "About secret scanning," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/code-security/secret-scanning

[12] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[13] Git Project, "git-log documentation," *Git Reference Manual*. [Online]. Available: https://git-scm.com/docs/git-log

[14] Git Project, "git-cat-file documentation," *Git Reference Manual*. [Online]. Available: https://git-scm.com/docs/git-cat-file

[15] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[16] P. Leach, M. Mealling, and R. Salz, "A universally unique identifier (UUID) URN namespace," IETF, RFC 4122, Jul. 2005.
[17] Amazon Web Services, "GetCallerIdentity," *AWS Security Token Service API Reference*. [Online]. Available: https://docs.aws.amazon.com/STS/latest/APIReference/API_GetCallerIdentity.html
[18] Amazon Web Services, "AWS Signature Version 4 for API requests," *IAM User Guide*. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_sigv.html
[19] GitHub, "REST API endpoints for users," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/rest/users/users
[20] JSON Schema, "JSON Schema draft 2020-12." [Online]. Available: https://json-schema.org/draft/2020-12
[21] W3C, "Content Security Policy level 3," W3C Working Draft. [Online]. Available: https://www.w3.org/TR/CSP3/
[22] GitHub, "REST API endpoints for search," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/rest/search/search
[23] OWASP Foundation, "Regular expression denial of service – ReDoS." [Online]. Available: https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS